

CHAPTER-4

ANALYZING FUNCTIONAL DEPENDENCIES, REDUNDANCY AND DATA ANOMALIES

INTRODUCTION

The Amazon Musical Instrument E-Commerce Management System is designed to efficiently manage customers, sellers, musical instruments, categories, brands, orders, payments, inventory, shipping, reviews, and wishlists. Functional dependencies help identify the relationships between attributes in each table, while redundancy analysis and data anomaly detection ensure data consistency and minimize duplication. A well-designed relational database improves performance, maintains data integrity, and supports efficient e-commerce operations.

1. CUSTOMER TABLE

Functional Dependency

Customer_ID $\rightarrow$ Customer_Name, Email, Phone_Number, Address, Password

Redundancy Analysis

Customer information is stored only once in the Customer table. Orders store only the Customer_ID, preventing duplicate customer records.

Data Anomalies

- **Insert Anomaly:** A new customer can be added without placing an order.
- **Update Anomaly:** Updating a customer's address or phone number requires changes only in the Customer table.
- **Delete Anomaly:** Deleting an order does not remove customer information.

2. CATEGORY TABLE

Functional Dependency

Category_ID $\rightarrow$ Category_Name

Redundancy Analysis

Each musical instrument category is stored only once. Products reference the category using Category_ID instead of repeating the category name.

Data Anomalies

- **Insert Anomaly:** New categories can be added without adding products.

- **Update Anomaly:** Renaming a category requires updating only one record.
- **Delete Anomaly:** Deleting a product does not remove the category.

3. BRAND TABLE

Functional Dependency

Brand_ID → Brand_Name

Redundancy Analysis

Brand information is maintained separately, avoiding repeated brand names for multiple products.

Data Anomalies

- **Insert Anomaly:** A new brand can be added before adding products.
- **Update Anomaly:** Updating the brand name is done only once.
- **Delete Anomaly:** Deleting a product does not delete the brand.

4. SELLER TABLE

Functional Dependency

Seller_ID → Seller_Name, Email, Phone_Number, Address

Redundancy Analysis

Seller details are stored only once. Products reference sellers using **Seller_ID**.

Data Anomalies

- **Insert Anomaly:** A seller can register without uploading products.
- **Update Anomaly:** Updating seller information affects only one record.
- **Delete Anomaly:** Deleting a product does not remove seller details.

5. PRODUCT TABLE

Functional Dependency

Product_ID → Product_Name, Price, Stock, Description, Category_ID, Brand_ID, Seller_ID

Redundancy Analysis

Product details are stored only in the Product table. Orders reference products using Product_ID, avoiding duplicate product information.

Data Anomalies

- **Insert Anomaly:** New products can be added before receiving orders.
- **Update Anomaly:** Product price or stock is updated only once.
- **Delete Anomaly:** Deleting an order does not remove product information.

6. ORDERS TABLE

Functional Dependency

Order_ID → Customer_ID, Order_Date, Total_Amount, Order_Status

Redundancy Analysis

The **Orders** table stores only order-related information. Customer details are referenced using **Customer_ID**.

Data Anomalies

- **Insert Anomaly:** Orders can be created after selecting a valid customer.
- **Update Anomaly:** Order status can be updated independently.
- **Delete Anomaly:** Deleting an order does not affect customer or product data.

7. ORDER_DETAILS TABLE

Functional Dependency

Order_Detail_ID → Order_ID, Product_ID, Quantity, Unit_Price

Redundancy Analysis

This table connects orders and products, preventing repeated product information in the Orders table.

Data Anomalies

- **Insert Anomaly:** Products can be added to an order independently.
- **Update Anomaly:** Quantity or price changes affect only the selected order item.
- **Delete Anomaly:** Removing an order item does not delete the product.

8. PAYMENT TABLE

Functional Dependency

Payment_ID → Order_ID, Payment_Method, Payment_Date, Payment_Status

Redundancy Analysis

Payment information is stored separately from the Orders table, avoiding duplicate payment records.

Data Anomalies

- **Insert Anomaly:** Payment details can be recorded after creating an order.
- **Update Anomaly:** Payment status is updated only once.
- **Delete Anomaly:** Deleting payment information does not remove the order.

9. INVENTORY TABLE

Functional Dependency

Inventory_ID → Product_ID, Stock_Quantity, Last_Updated

Redundancy Analysis

Inventory details are stored separately from the Product table, making stock management efficient.

Data Anomalies

- **Insert Anomaly:** Inventory records can be created when new products are added.
- **Update Anomaly:** Stock quantity is updated only in the Inventory table.
- **Delete Anomaly:** Removing inventory records does not delete product information.

10. SHIPPING TABLE

Functional Dependency

Shipping_ID → Order_ID, Delivery_Address, Tracking_Number, Delivery_Status

Redundancy Analysis

Shipping details are maintained separately, preventing repeated delivery information.

Data Anomalies

- **Insert Anomaly:** Shipping records can be created after order confirmation.

- **Update Anomaly:** Delivery status can be updated independently.
- **Delete Anomaly:** Deleting shipping information does not affect order details.

11. REVIEW TABLE

Functional Dependency

Review_ID → Customer_ID, Product_ID, Rating, Comment, Review_Date

Redundancy Analysis

Customer reviews are stored separately from products, avoiding duplicate review data.

Data Anomalies

- **Insert Anomaly:** Customers can submit reviews after purchasing products.
- **Update Anomaly:** Reviews can be edited without affecting product information.
- **Delete Anomaly:** Deleting a review does not remove product records.

12. WISHLIST TABLE

Functional Dependency

wishlist_ID → Customer_ID, Product_ID

Redundancy Analysis

Wishlist items are stored separately, allowing customers to save products without duplicating product information.

Data Anomalies

- **Insert Anomaly:** Customers can add products to their wishlist without placing an order.
- **Update Anomaly:** Wishlist items can be modified independently.
- **Delete Anomaly:** Removing a wishlist item does not delete the product or customer information.

CONCLUSION

The **Amazon Musical Instrument E-Commerce Management System** is designed using proper functional dependencies and normalized tables. Each entity stores only its relevant information, reducing redundancy and preventing **insert, update, and delete anomalies**.

This database design ensures **data consistency, integrity, security, and efficient management** of customers, products, orders, inventory, payments, and shipping operations, making it suitable for a real-world e-commerce platform like Amazon.